Escuela de Ingeniería Electrónica.

Taller de Diseño Digital.

Documentación laboratorio #1

Tema:

Introducción al diseño digital con HDL y herramientas EDA de síntesis.

Profesor:

Kaled Alfaro Badilla.

Estudiantes:

María Paulina Camacho Herrera – 2021093601

Kendall Jesús Barrantes Chaves-2021442238

Luis Daniel Zumbado Sáenz - 2018176234

José Pablo Herrera Vagas - 2023029239

I Semestre 2026

En el presente documento se realizará una explicación detallada de cada uno de los ejercicios propuestos para el primer proyecto del curso taller de diseño digital, para cada uno de estos se brindará información de cada uno de los módulos para cada ejercicio, su respectivo banco de pruebas para verificar el funcionamiento del código mediante simulación, así como s archivo de restricciones para comprobar el código implementado mediante una FPGA. Adicionalmente se mostrará un diagrama de bloques para cada diseño y una tabla de verdad que refleje todas las combinaciones posibles para cada circuito lógico.

## Ejercicio#1

### Source

Para el primer ejercicio se realizó un módulo que recibe 4 bits desde la entrada representando los interruptores y muestra en las salidas los LEDS que representaran su negativo en complemento a 2, dicha entrada se representa con un vector de 4 bits denominado sw lo que representa los interruptores en la FPGA mientras que la salida es un vector de 4 bits de salida denominado led, representando los leds de salida en la FPGA.

La función assing led= -sw calcula el complemento a 2 del vector de entrada y lo asigna en el vector led, por lo que dependiendo de que interruptor este encendido se encenderán diferentes LEDS para visualizar el complemento a 2.

### Diagrama de bloques

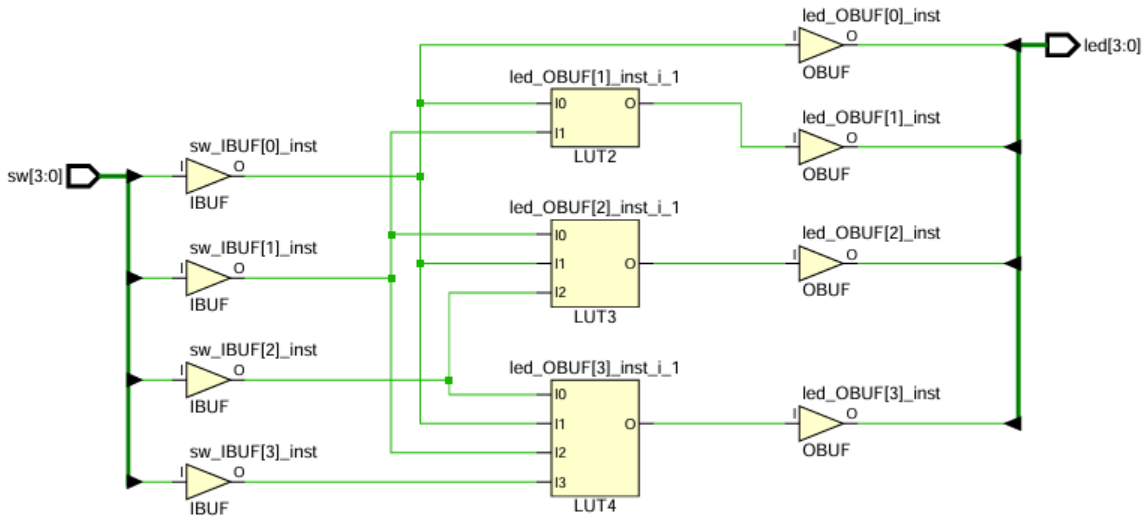


**Fig.1.** Diagrama de bloques correspondiente al ejercicio 1.

**Testbench**

Para iniciar el testbench se conectan las señales de este con el módulo real, para realizar la simulación el bucle de prueba recorre todas las combinaciones posibles de 4 bits (0-15) por lo que dependiendo del valor en binario que representen las entradas se calculara el complemento a 2 de cada número binario, si en la simulación se detectan fallos se mostrara un mensaje de error.

**Constrain**

El archivo de restricciones asigna un pin físico correspondiente a cada señal del módulo, los switches están asignados al vector de entrada sw, mientras que los LEDS en la FPGA son asignados a la salida del módulo.

**Tabla de verdad Ejercicio #1**

| Sw [3:0] | Led [3:0] | Entrada (Decimal) | Salida (Decimal) |
|---|---|---|---|
| 0000 | 0000 | 0 | 0 |
| 0001 | 1111 | 1 | -1 |
| 0010 | 1110 | 2 | -2 |
| 0011 | 1101 | 3 | -3 |
| 0100 | 1100 | 4 | -4 |
| 0101 | 1011 | 5 | -5 |
| 0110 | 1010 | 6 | -6 |
| 0111 | 1001 | 7 | -7 |
| 1000 | 1000 | -8 | -8 |
| 1001 | 0111 | -7 | 7 |
| 1010 | 0110 | -6 | 6 |
| 1011 | 0101 | -5 | 5 |
| 1100 | 0100 | -4 | 4 |
| 1101 | 0011 | -3 | 3 |
| 1110 | 0010 | -2 | 2 |
| 1111 | 0001 | -1 | 1 |

**<u>Ejercicio#2</u>**

**Source**

Para este ejercicio se implementa un multiplexor de cuatro entradas parametrizado en verilog, al iniciar el módulo se define un parámetro WIDTH representando el ancho de bits, en este caso 4, esto determina cuantos bits tiene cada una de las entradas y salidas de datos, el módulo contiene cuatro entradas de datos denominadas d0,d1,d2,d3 y cada una contiene un ancho de bits, el módulo también contiene otra entrada definida como sel, esta representa un selector de 2 bits, en este, se pueden asignar 4 combinaciones posibles (00,01,10,11).

La salida se asigna como y el cual tomara un valor distinto de salida dependiendo del valor indicado en el selector, decidiendo cual entrada se conecta a la salida, el código incluye una opción default para evitar latches, para este caso a la salida se le asignara un 0. El código de este multiplexor se implementa dentro de un bloque always lo que indica que este bloque debe ejecutarse cada vez que se realice un cambio a la entrada del multiplexor, dentro de este bloque se utiliza un case, el cual evalúa el valor de selector.

**Diagrama de bloques**

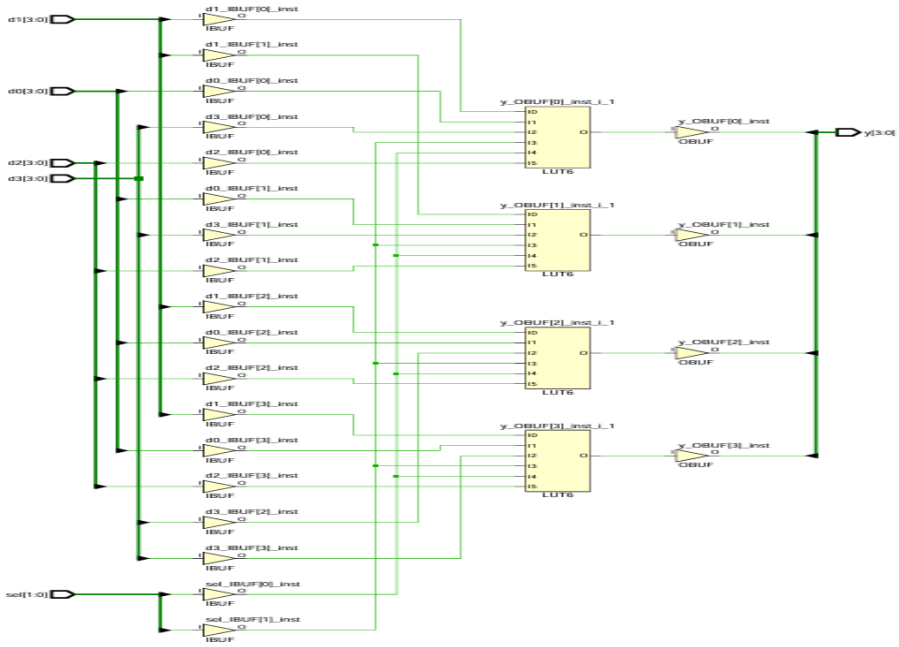


**Fig.2.** Diagrama de bloques correspondiente al ejercicio 2.

**Testbench**

Para la verificación y simulación del multiplexor antes de la implementación en la FPGA se generarán señales de prueba para comparar la salida obtenida con la esperada y verificar si funciona correctamente y si no existen errores, este testbench automatiza la verificación del multiplexor generando múltiples combinaciones aleatorias de entradas y comparando la salida del circuito con el valor esperado, al iniciar el banco de pruebas se define un parámetro que indica el ancho de banda de las señales que se van a probar, este valor se asigna como uno de 16 bits, aunque este valor puede ser modificado.

Se declaran las señales de interconexión para conectar el banco de pruebas con el módulo principal, conectando las señales de entrada y salida del multiplexor, además, se declara el valor expected_y el cual almacena el valor que aparece en la salida si este se asigna de manera correcta, se asigna también una variable errors y una variable i que funciona como contador

para enumerar las pruebas que se realizaran, posteriormente se instancia el módulo con el parámetro actual.

Para verificar de forma automática el resultado obtenido se define una tarea llamada check_result, esta tarea verifica si la salida del multiplexor contiene un valor correcto por medio de la estructura case, posteriormente, se introduce un retraso de tiempo para actualizar la salida antes de cambiar la entrada, si la salida en y y en expected_y no coinciden la simulación indica un error mostrando el número de interacción el valor del selector y las salidas obtenidas.

Toda la simulación se realizará por medio de un ciclo for que repetirá las pruebas 50 veces donde en cada iteración se generaran valores aleatorios para las entradas y un valor aleatorio entere 0 y 3 para el selector, al finalizar la simulación el sistema indica si si la simulación fue exitosa y se ejecuta $stop para detener la operación.

**Constrain**

El archivo de restricciones para este ejercicio indica los interruptores, botones y leds para las entradas y salidas del multiplexor, primero se configuran las entradas a los switches del 0 al 3 de la nexys 4, cada archivo usa el comando set_property para indicar el pin físico de la FPGA, luego se asignan las conexiones correspondientes a d0,d1,d2 y d3, cada bit de cada entrada se conecta directamente a un pin diferente de la FPGA, De esta forma cuando mueves los interruptores, estas cambiando el valor de las entradas, si mueves los switches del 0 al 3, varia el valor de d0, del 4 al 7 varia d1, del 8 al 11 varias d2 y del 12 al 15 varia d3.

Posteriormente se define el selector que utiliza los botones de la tarjeta en donde en bit sel[0] se conecta a BTNR (botón derecho) y sel[1] a BTNL (botón izquierdo), finalmente las salidas se conectan a los leds de la tarjeta, cuando se selecciona una entrada mediante los botones del selector el valor de la entrada se mostrara en los LEDS de salida.

Tabla de verdad Ejercicio #2

| **sel1** | **sel0** | **Salida y** |
|---|---|---|
| 0 | 0 | d0 |
| 0 | 1 | d1 |
| 1 | 0 | d2 |
| 1 | 1 | d3 |

Interpretación:

- sel = 00 → la salida toma el valor de d0
- sel = 01 → la salida toma el valor de d1
- sel = 10 → la salida toma el valor de d2
- sel = 11 → la salida toma el valor de d3 Como el módulo es parametrizado, cada entrada y la salida tienen un ancho de WIDTH bits (4, 8 o 16).

**<u>Ejercicio #3</u>**

**Source**

Este código genera una señal PWM para controlar el brillo de un LED usando interruptores, este módulo tiene como entradas un reloj(clk) un botón de reset y 4 switches que controlan el ciclo de trabajo, la única salida es la señal PWM que viaja hacia el LED, el código cuenta con un contador principal de 16 bits capaz de contar hasta 100000 repeticiones.

Para ejecutar continuamente la se señal se crea un generador de rampa, que inicia con el contador en 0 y por cada flanco positivo del reloj aumenta el contador, cuando este llega a 99999 este vuelve a 0, para generar el ancho de pulso se crea un comparador para que ocurra la modulación, si la señal pwm_out es 1'b1 esta estará encendida, si es 1'b0 estará apagada, la cantidad de ciclos de reloj para un ciclo de trabajo dentro del periodo del PWM es de 6250 ya que el periodo total es de 100000 ciclos y los switches contienen 4 bits por lo que se pueden representar 16 valores, entonces:

$$100000/16=6250$$

El tiempo para que PWM permanezca en 1 depende del valor de switch, entre más alto sea el valor del switch más tiempo se mantendrá el LED encendido.

**Diagrama de bloques**

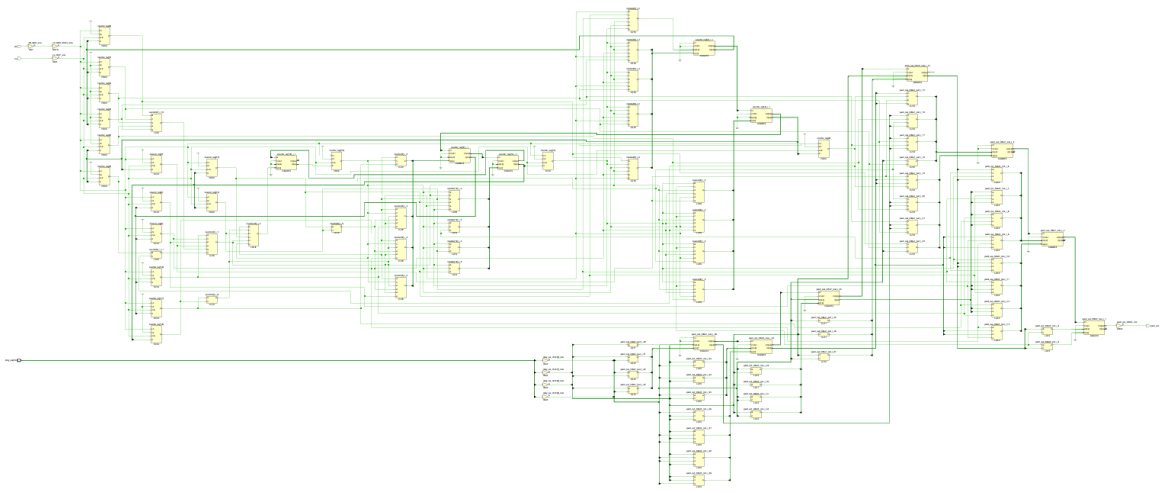

**Fig.3.** Diagrama de bloques correspondiente al ejercicio 3.

## Testbench

Para la simulación del funcionamiento del módulo se definen las señales de prueba en el testbench: clk, rst, duty_sw, pwm_out (mismas entradas y salidas del módulo original, posteriormente, se instancia el módulo conectando cada una dea estas señales para observar la respuesta de la simulación,

Se genera un reloj artificial utilizando un bloque initial, este reloj cambia cada 5ns por lo que se produce un periodo total de 10ns, en otro bloque initial se define la secuencia de pruebas donde se activa el reset durante 100ns y posteriormente se desactiva, justo para que comience la validación del PWM.

Para esta simulación se evaluación 4 casos principales, el primero se asigna un duty_sw=0 por lo que el ciclo de trabajo debe ser 0 por lo que la salida debe mantenerse en 0, con duty_sw=4 se tiene un 25% de ciclo de trabajo aproximadamente, por lo que en este caso la señal de salida debe estar en alto una cuarta parte del periodo, para duty_sw=8 tenemos un 50% de ciclo de trabajo y la salida debe ser una señal cuadrada perfecta, para el último caso duty_sw=15 por que contiene casi todo el ciclo de trabajo del generador por lo que la señal debe permanecer en alto la mayor parte del tiempo.

## Constrain

Para conectar el diseño en verilog con los pines de la FPGA, se configura la senal de reloj y se asigna al pin E3, además, se especifica el estándar LVCMOS33 para que la senal trabaje a 3.3V. Posteriormente, se definen los 4 switches que controlan el ciclo de trabajo en donde cada bit del vector se conecta a un interruptor físico distinto en la NEXYS 4, los pines utilizados correspondes para duty_sw(0), duty_sw(1), duty_sw(2), duty_sw(3) son sucesivamente J15, L16, M13, R15, esto para que el usuario cambie manualmente el valor de ciclo de trabajo mediante los switches, si todos los switches están apagados (0000) el led no encenderá y el ciclo de trabajo sera de 0% mientras que si todos están encendidos (1111) el ciclo de trabajo será casi del 100%, el valor del ciclo de trabajo dependerá del valor en binario de los interruptores apagados o encendidos.

Se asigna un botón de reset al pin N17 para reiniciar el contador del pulso, la salida esta asociada al pin H17 el cual representa un LED en la FPGA, por lo que este se encandera o apagara dependiendo de la señal PWM que se asigen dependiendo de los interruptores.

**Tabla de verdad**

| duty_sw | Ciclo de trabajo aproximado | Comportamiento de pwm_out |
|---|---|---|
| 0000 | 0% | Siempre en 0 |
| 0001 | 6.25% | Alto durante 6250 ciclos, bajo el resto |
| 0010 | 12.5% | Alto durante 12500 ciclos, bajo el resto |
| 0011 | 18.75% | Alto durante 18750 ciclos, bajo el resto |
| 0100 | 25% | Alto durante 25000 ciclos, bajo el resto |
| 0101 | 31.25% | Alto durante 31250 ciclos, bajo el resto |
| 0110 | 37.5% | Alto durante 37500 ciclos, bajo el resto |
| 0111 | 43.75% | Alto durante 43750 ciclos, bajo el resto |
| 1000 | 50% | Alto durante 50000 ciclos, bajo el resto |
| 1001 | 56.25% | Alto durante 56250 ciclos, bajo el resto |
| 1010 | 62.5% | Alto durante 62500 ciclos, bajo el resto |
| 1011 | 68.75% | Alto durante 68750 ciclos, bajo el resto |
| 1100 | 75% | Alto durante 75000 ciclos, bajo el resto |
| 1101 | 81.25% | Alto durante 81250 ciclos, bajo el resto |
| 1110 | 87.5% | Alto durante 87500 ciclos, bajo el resto |
| 1111 | 93.75% | Alto durante 93750 ciclos, bajo el resto |

**Regla de funcionamiento La salida PWM se define así:**

- pwm_out = 1 si counter < duty_sw × 6250
- pwm_out = 0 en cualquier otro caso.

**Ejercicio4**

**Source**

En este ejercicio, creamos un sistema que permite a la FPGA enviar el mensaje “Hola mundo” y recibir datos desde la computadora para mostrarlos en los LEDs.

El módulo principal tiene algunas entradas importantes. Estas son el reloj de 100 MHz, un reset activo en bajo llamado rst_n, un botón llamado btn_send que arranca la transmisión y la señal serial de entrada rx_pin. Como salidas, utilizamos la línea de transmisión tx_pin y un bus de 8 bits llamado leds.

Para evitar que una sola pulsación del botón active la transmisión varias veces, usamos un detector de flanco. Este bloque genera una señal interna llamada btn_pressed solo cuando detecta una transición de 0 a 1 en btn_send.

El sistema se divide en tres bloques principales:

- UART TX: se encarga de transmitir datos seriales por tx_pin a 9600 baudios. Recibe una señal tx_start para comenzar el envío y un dato tx_data con el carácter que se quiere transmitir.
- UART RX: recibe la información por rx_pin y, cuando identifica una trama válida, entrega el byte en rx_data_out junto con la señal rx_done.
- FSM o message_sender: coordina el envío del mensaje.

Para ello utiliza un contador llamado char_ptr y una pequeña ROM implementada con un case, donde cada posición corresponde a un carácter ASCII del texto “Hola mundo".

La FSM comienza en el estado IDLE, donde espera a que el usuario presione el botón. Después pasa al estado SEND, en el que activa tx_start para enviar el carácter actual.

Luego entra en el estado WAIT, donde espera a que el transmisor indique que terminó mediante la señal tx_ready. Este ciclo se repite hasta completar el envío de todos los caracteres del mensaje.

Además, cuando el receptor detecta un dato válido, ese valor se guarda en el registro de salida leds. Esto permite visualizar en la FPGA el byte recibido en formato binario.

## Diagrama de bloques

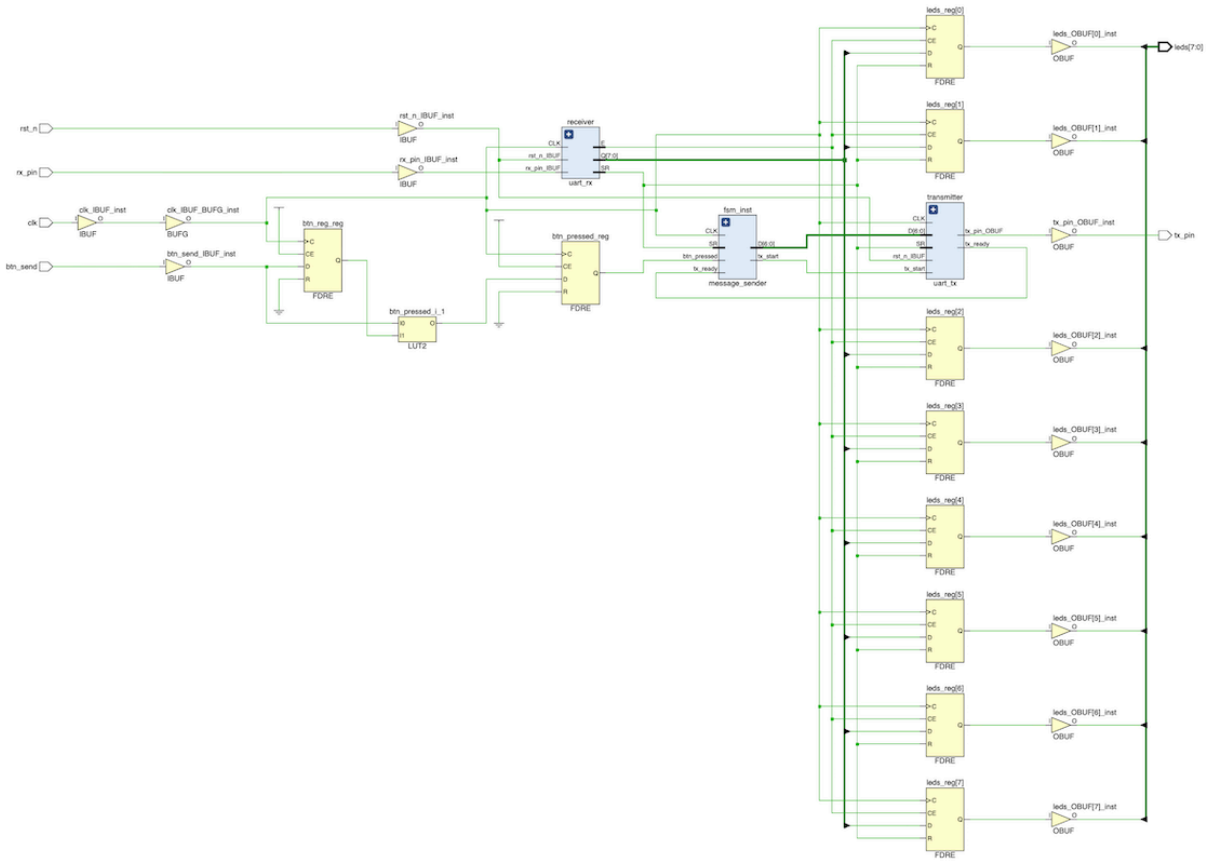


**Fig.4.** Diagrama de bloques correspondiente al ejercicio 4.

## Testbench

El banco de pruebas fue diseñado para probar cómo funciona el sistema UART. Primero, se crean las señales de prueba y luego se instancia el módulo principal.

Se genera un reloj de 100 MHz con un bloque inicial que tiene un período de 10 ns. Luego, se hace un reinicio inicial manteniendo rst_n en cero durante un momento y luego se activa para que el sistema funcione normalmente.

La primera prueba es simular que se presiona el botón btn_send. Esto debe activar el detector de flanco y empezar a transmitir el mensaje. Para verificar esto, el testbench espera a que la señal tx_pin cambie de alto a bajo, lo que indica el inicio de una trama UART, es decir, el bit de inicio.

La segunda prueba se centra en recibir datos. En este caso, se simula una trama UART que corresponde a la letra 'A' (0x41). Los bits se envían por rx_pin siguiendo el tiempo de bit para 9600 baudios. Primero, se envía el bit de inicio, luego los 8 bits de datos en formato LSB-first, y al final el bit de parada.

Después de enviar la trama, se espera a que el módulo receptor reconstruya correctamente el byte recibido y que ese valor se muestre en los LEDs. De esta forma, el testbench permite probar tanto la parte de transmisión como la de recepción del sistema UART.

**Constrain**

El archivo de restricciones es fundamental para relacionar el diseño en Verilog con los recursos físicos de la FPGA Nexys 4. En él, se asigna cada señal a un pin específico y se define el estándar eléctrico correspondiente.

La señal de reloj, (clk), se conecta al pin del oscilador de 100 MHz de la tarjeta. Este oscilador es el encargado de sincronizar todo el sistema. La señal de reset, (rst_n), se asigna a un botón físico. De esta manera, el usuario puede reiniciar el sistema y regresar la FSM a su estado inicial. También se apagan los LEDs.

El botón (btn_send) se conecta a otro pulsador de la tarjeta. Al presionarlo, se inicia la transmisión del mensaje.

Las señales (tx_pin) y (rx_pin) se asignan a pines físicos para la comunicación serial. Esto permite enlazar la FPGA con la computadora mediante un adaptador UART.

Finalmente, el bus leds [7:0] se conecta a los LEDs de la tarjeta. De esta forma, el dato recibido por UART se puede visualizar directamente en la FPGA.

**Tabla funcional de sistema.**

| Entrada / condición | Salida / acción |
|---|---|
| rst_n = 0 | El sistema se reinicia, leds = 0, la FSM vuelve a IDLE |
| Flanco de subida en btn_send | Se activa btn_pressed y comienza el envío del mensaje |
| btn_pressed = 1 en estado IDLE | La FSM pasa a SEND |
| Estado SEND | Se coloca en tx_data el carácter actual y se activa tx_start |
| tx_start = 1 | El transmisor UART inicia el envío por tx_pin |
| tx_ready = 1 | El transmisor terminó de enviar un carácter |
| tx_ready = 1 y aún faltan caracteres | La FSM incrementa char_ptr y envía el siguiente carácter |
| tx_ready = 1 y char_ptr = 9 | Se termina de enviar "Hola mundo" y la FSM vuelve a IDLE |
| rx_done = 1 | El dato recibido rx_data_out se carga en leds |
| Trama UART válida recibida por rx_pin | El receptor entrega el byte en rx_data_out |

**Tabla del mensaje Transmitido**

| char_ptr | tx_data (hex) | Carácter |
|---|---|---|
| 0 | 48h | H |
| 1 | 6Fh | oo |
| 2 | 6Ch | l |
| 3 | 61h | a |
| 4 | 20h | espacio |
| 5 | 6Dh | m |
| 6 | 75h | u |
| 7 | 6Eh | n |
| 8 | 64h | d |
| 9 | 6Fh | o |

**Tabla funcional del multiplexado.**

| Condición | Valor de leds |
|---|---|
| rst_n = 0 | 00000000 |
| rx_done = 1 | leds = rx_data_out |
| En otro caso | leds conserva su valor anterior |